

Архитектура мобильного приложения магазина мебели с AR-примеркой

Что внутри

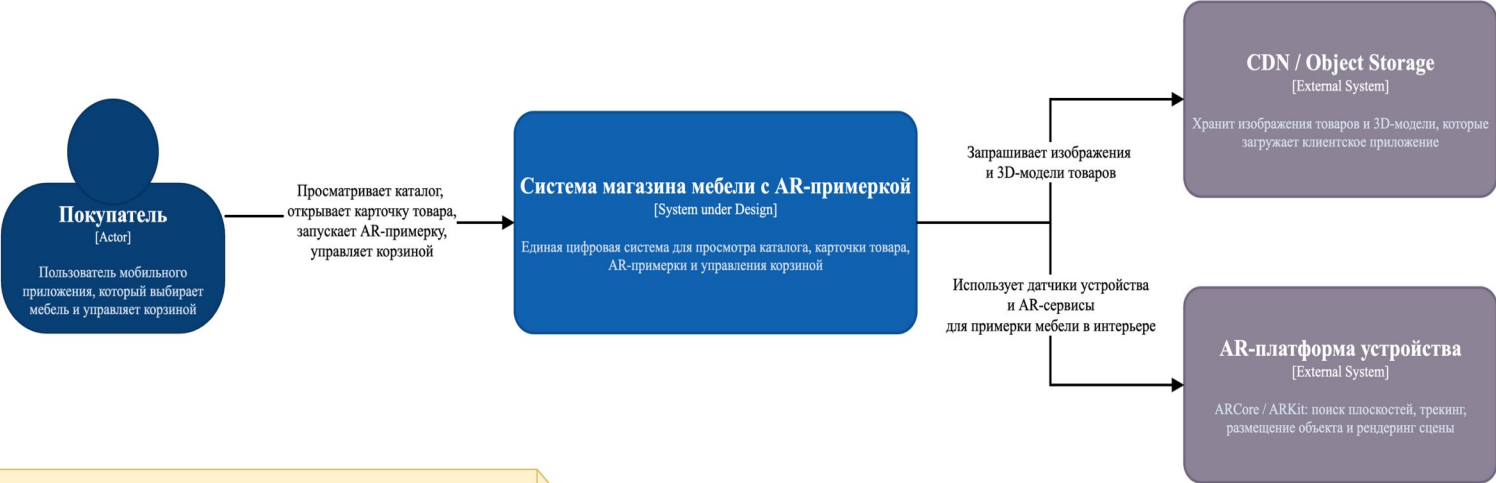
- C4 Context: кто использует систему и с какими внешними системами она взаимодействует
- C4 Container: из каких технических контейнеров состоит решение
- Domain Model: какие бизнес-сущности и value objects лежат в основе предметной области
- UML Core + Scaled Architecture: как реализован ключевой PDP-AR-сценарий и как ядро связано с инфраструктурой
- Facade Diagram: как скрыта сложность iOS AR-подсистемы

C4 Context

Кто взаимодействует с системой и какие внешние зависимости есть у проекта

C4 Context Diagram

Задание 1 + домашняя доработка: акторы, внешние системы, use cases и peer review



Акторы, внешние системы и ключевые Use Cases

Актор: Покупатель.
Внешние системы: CDN / Object Storage, AR-платформа устройства (ARCore / ARKit).
Use Case 1: Покупатель просматривает листинг товаров.
Use Case 2: Покупатель открывает карточку товара и изучает характеристики.
Use Case 3: Покупатель загружает 3D-модель и запускает AR-примерку.
Use Case 4: Покупатель добавляет товар в корзину и меняет количество.

Описание Для ПЗ

Контекстная диаграмма показывает систему магазина мебели с AR-примеркой как единый программный продукт. Основным пользователем является покупатель, который просматривает каталог, изучает карточки товаров, запускает AR-сценарий и управляет составом корзины. Для реализации пользовательских сценариев система взаимодействует с внешним CDN / Object Storage, из которого получает изображения и 3D-модели товаров, а также с AR-платформой устройства, обеспечивающей поиск плоскостей, трекинг и размещение виртуальных объектов. Технологические детали намеренно скрыты, так как они относятся к контейнерному уровню C4.

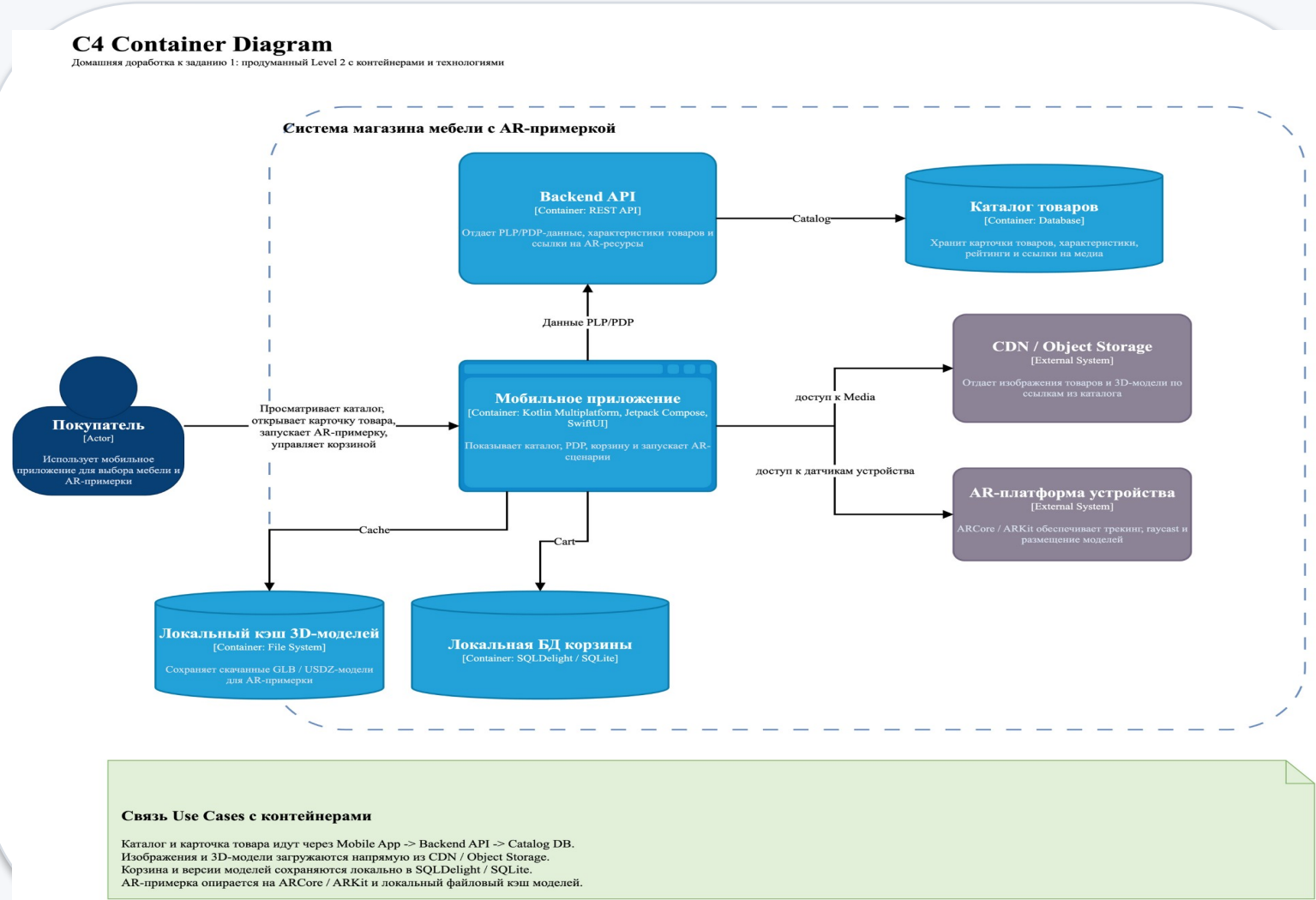
Что показывает диаграмма

- Система рассматривается как единый программный продукт магазина мебели с AR-примеркой.
- Основной актор: покупатель, который просматривает каталог, открывает карточку товара, запускает AR и управляет корзиной.
- Внешние системы: CDN/Object Storage для медиа и 3D-моделей, а также ARCore/ARKit как платформенные AR-сервисы устройства.

Эта схема нужна, чтобы зафиксировать границы системы и ключевые пользовательские use cases до обсуждения технологий.

C4 Container

Из каких технических контейнеров состоит решение и где живут данные



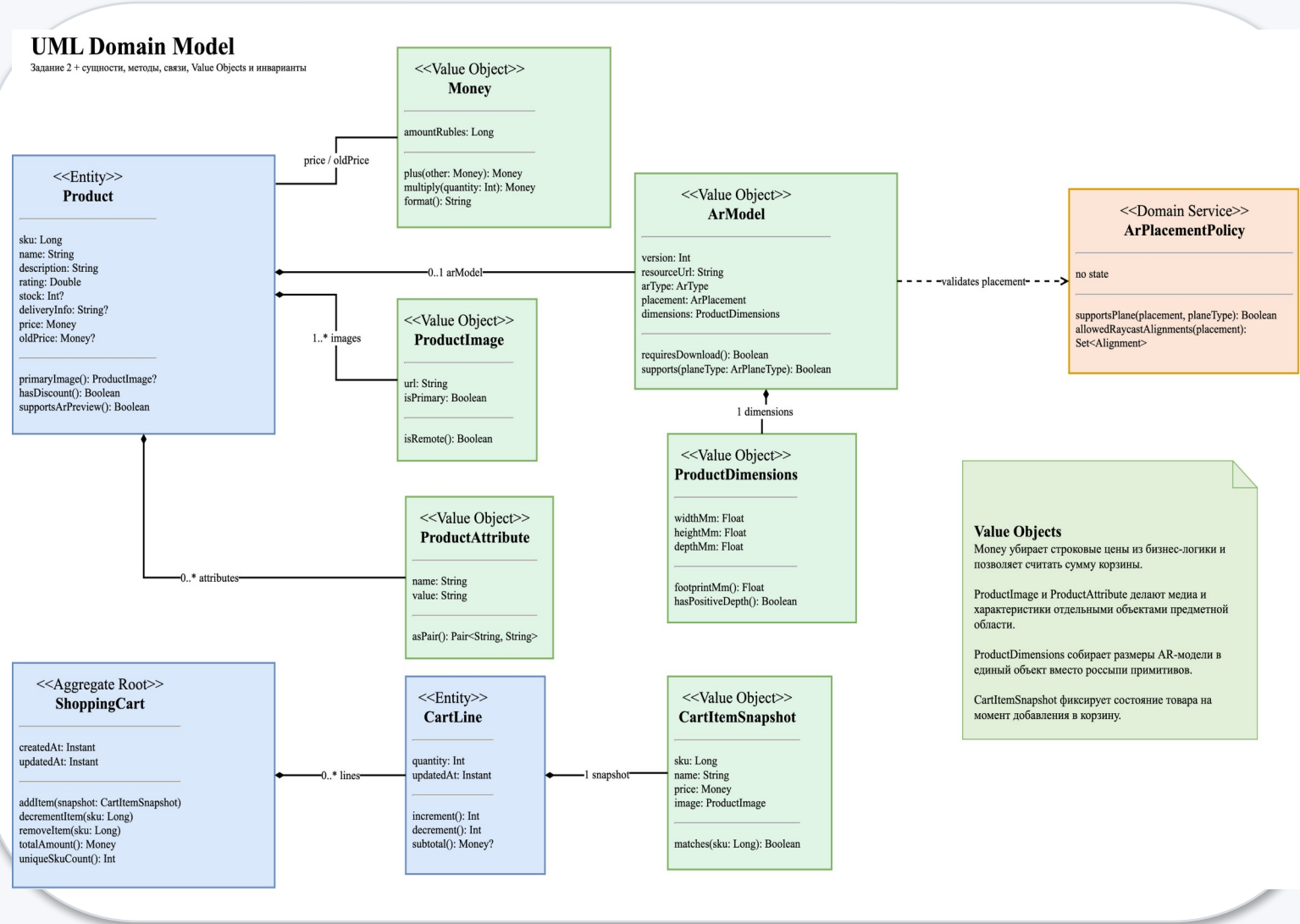
Что показывает диаграмма

- Пользователь работает через мобильное приложение, которое получает данные каталога через Backend API.
- Карточки товаров и ссылки на медиа живут в каталоге, а корзина и metadata моделей хранятся локально.
- AR-сценарий опирается на локальный файловый кэш 3D-моделей и на платформенные сервисы ARCore/ARKit.

Эта схема раскрывает систему до уровня контейнеров и показывает, как пользовательские сценарии распределяются по техническим блокам.

Domain Model

Какие бизнес-сущности лежат в основе предметной области проекта



Что показывает диаграмма

- Центральные сущности: Product и ShoppingCart; CartLine живет внутри корзины как часть агрегата.
- Value Objects выделены отдельно: Money, ProductImage, ProductAttribute, ArModel, ProductDimensions, CartItemSnapshot.
- Смысл диаграммы не в БД, а в бизнес-логике: поведении сущностей, связях и инвариантах предметной области.

Здесь фиксируется доменная модель без SQL-типов и таблиц: сначала сущности и их смысл, потом уже возможный ORM-маппинг.

UML Core Domain: PDP AR Flow

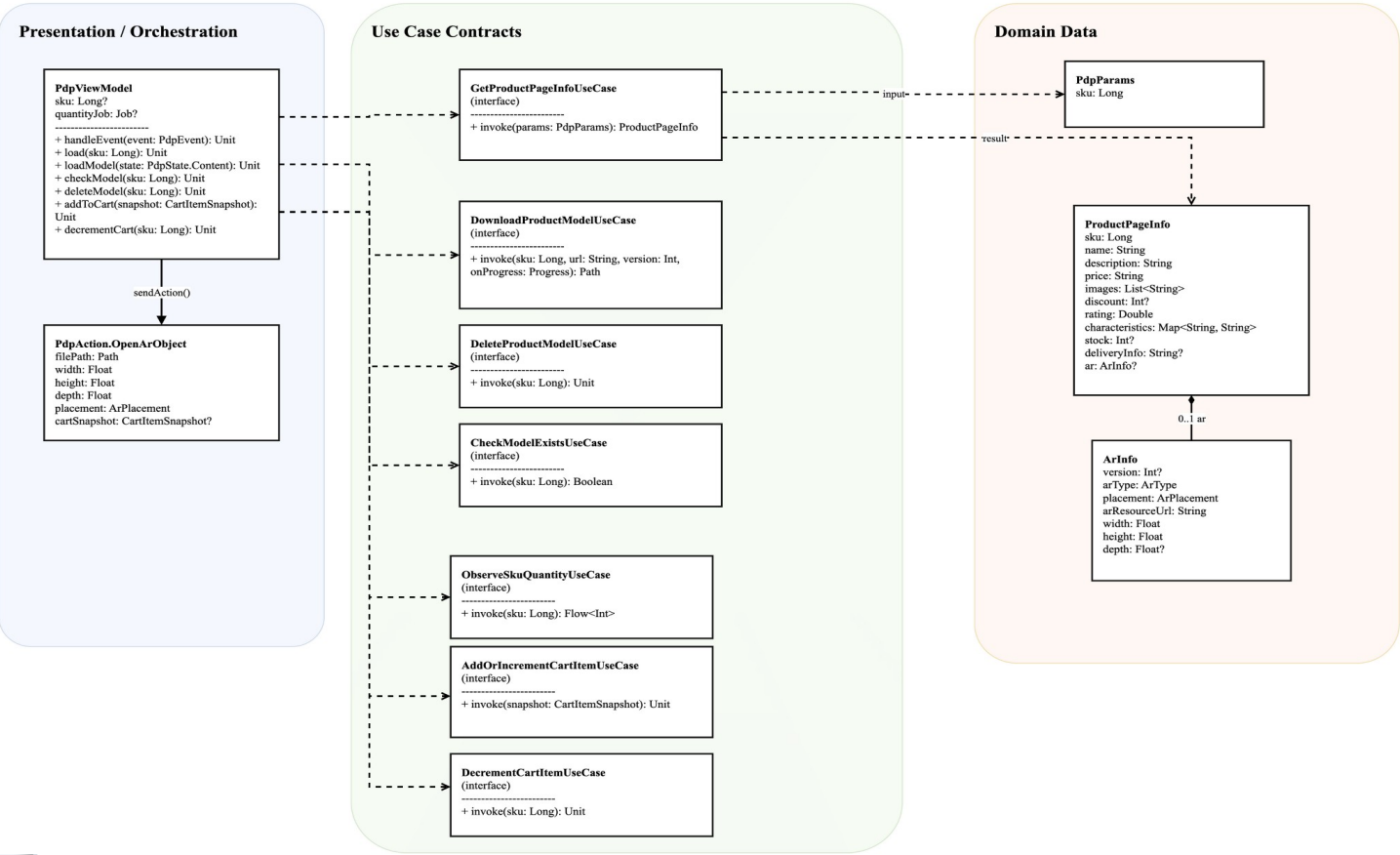
Ключевой сценарий внутри ядра приложения

UML Core Domain - PDP AR Flow

Блок 3. Ключевой сценарий: карточка товара - кэш 3D-модели - запуск AR - синхронизация корзины

Смысл сценария

Пользователь открывает PDP, приложение загружает ProductPageInfo, проверяет локальный кэш 3D-модели, при необходимости скачивает файл, затем формирует одноразовое действие OpenArObject и одновременно поддерживает количество товара в локальной корзине.



Что показывает диаграмма

- PdpViewModel оркестрирует сценарий карточки товара и знает только use case-контракты.
- Сценарий: загрузить ProductPageInfo, проверить локальный кэш модели, при необходимости скачать 3D-файл и выпустить OpenArObject.
- Параллельно через отдельные use cases поддерживается наблюдение и изменение количества товара в локальной корзине.

Эта схема показывает core-логику ключевого AR-сценария без привязки к конкретной инфраструктуре.

Scaled Architecture

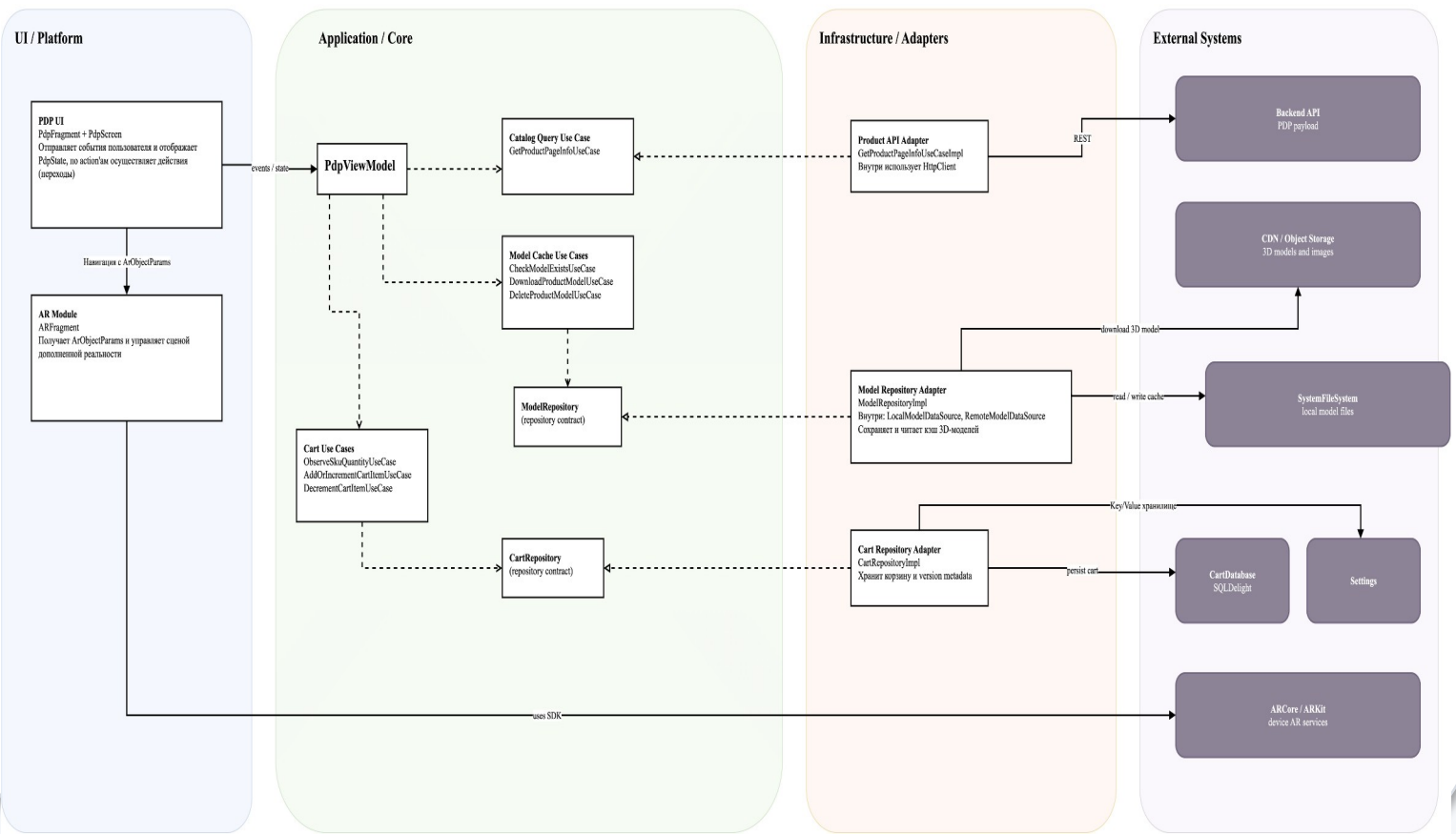
Как UI, core и инфраструктура соединяются в общем архитектурном контуре

Масштабирование Ядра: UI и Infrastructure

Домашняя доработка к блоку 3: упрощенная модульная схема без перегрузки внутренними страницами

Идея

На этой странице скрыты внутренние DataBosco-классы. Показаны только границы слоев, контракты ядра и адаптеры, которые реализуют эти контракты.



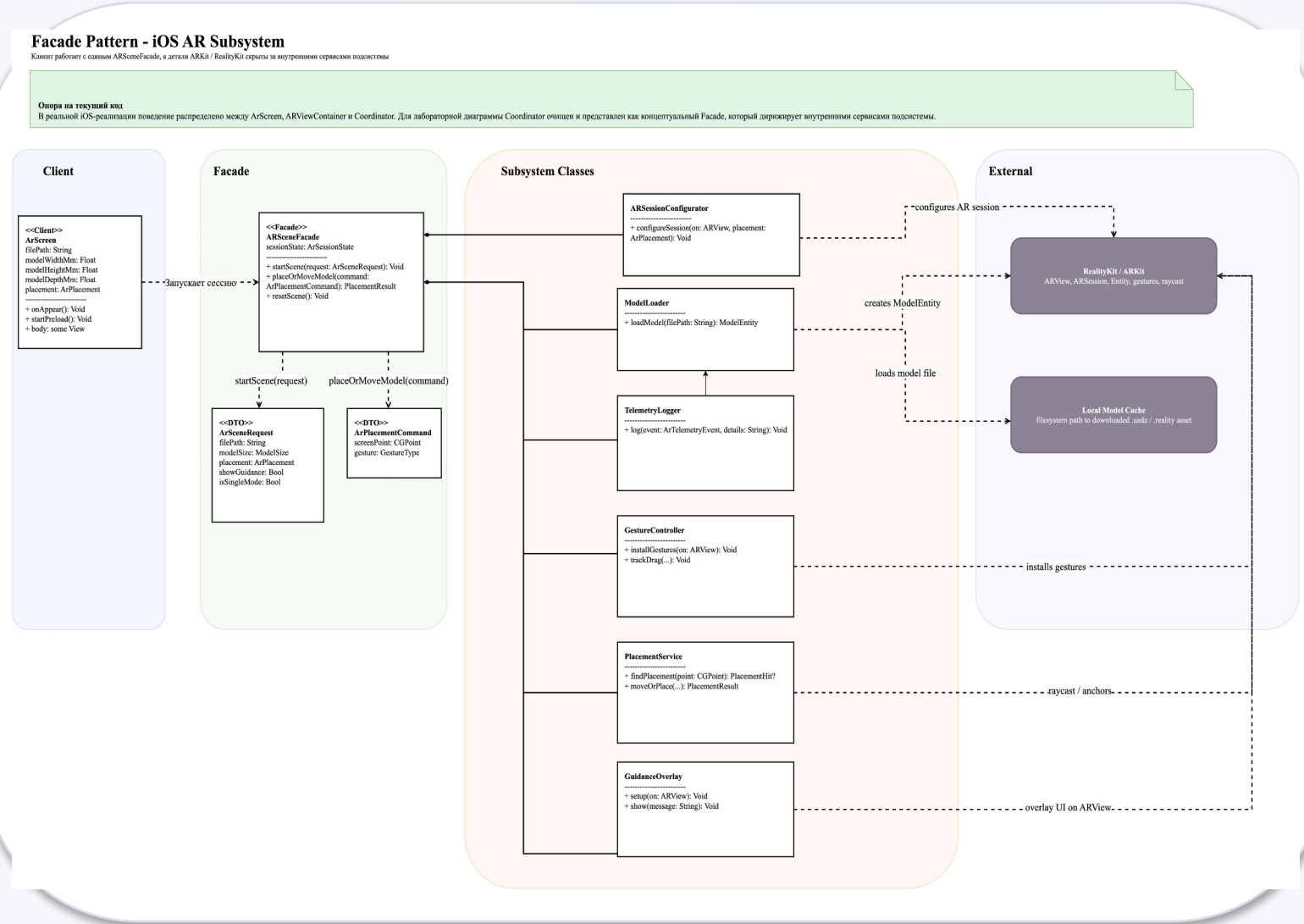
Что показывает диаграмма

- Слева расположен UI, в центре core со ViewModel, use cases и repository contracts, справа adapters и внешние системы.
- UI не ходит напрямую в сеть, БД или файловую систему: все идет через ViewModel и use case-контракты.
- Диаграмма иллюстрирует Clean Architecture, DIP и возможность менять concrete-реализации без переписывания ядра.

Главная мысль: зависимости направлены к абстракциям, а инфраструктура подключается как внешний слой через adapters.

Facade Pattern: iOS AR Subsystem

Как скрыта сложность AR-подсистемы на iOS



Что показывает диаграмма

- Клиентом выступает ArScreen, который работает не с ARKit напрямую, а с единой точкой входа ARSceneFacade.
- Facade координирует внутренние сервисы: конфигурацию сессии, загрузку модели, размещение объекта, жесты, overlay и telemetry.
- Внешняя сложность RealityKit/ARKit и локального model cache скрыта за коротким бизнес-интерфейсом фасада.

Эта схема показывает применение структурного паттерна Facade для упрощения сложной AR-подсистемы.

Итог

Логика проектирования от бизнес-уровня к реализации

Главные выводы

- Контекстная и контейнерная диаграммы задают границы системы и показывают техническое разбиение решения.
- Доменная модель фиксирует устойчивые бизнес-сущности: товар, корзину и AR-описание продукта.
- Core-диаграмма показывает ключевой пользовательский сценарий PDP -> кэш модели -> AR -> корзина.
- Scaled Architecture объясняет, как UI, use cases, repository contracts и adapters связаны по правилам Clean Architecture.
- Facade Diagram демонстрирует локальное упрощение сложной iOS AR-подсистемы через единый ARSceneFacade.

Вся серия диаграмм описывает один и тот же проект на разных уровнях абстракции: сначала внешнее окружение, затем внутреннюю структуру, потом доменную модель, ключевой сценарий и частный архитектурный паттерн.

Такой набор схем можно использовать и в презентации, и в пояснительной записке: каждая диаграмма отвечает на свой архитектурный вопрос и не дублирует предыдущую.